14 haftalık yol haritasında 4. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 04

Operatörler, İfadeler ve Tip Dönüşümleri: ALU'nun Dili

İşlemci veriyi nasıl hesaplar, C bir ifadeyi hangi sırayla çözer ve tip değiştirirken bitlere ne olur?

MODÜL II · VERİNİN TEMSİLİ VE İŞLENMESİ · H3–H4

> HAFTANIN BÜYÜK SORUSU
>
> *Tek bir tip dönüşümü, yüz milyonlarca dolarlık bir roketi nasıl havada parçalayabilir?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ... 3
1.1 Öğrenme çıktıları ... 3
2. Geçen Haftadan: Değişkenlerden İfadelere ... 4
3. Kavramsal Anlatım ... 4
3.1 İfade, işlenen ve operatör: ALU'nun dili ... 4
3.2 Aritmetik operatörler: tamsayı bölmesi ve kalan ... 5
3.3 Atama ve bileşik atama ... 5
3.4 Artırma ve azaltma: önek ve sonek ... 5
3.5 İlişkisel operatörler ve C'de doğruluk değeri ... 6
3.6 Mantıksal operatörler ve kısa devre değerlendirme ... 6
3.7 Bit düzeyi operatörler ... 7
3.8 Öncelik, birleşme yönü ve ifade ağaçları ... 8
3.9 Yan etkiler, sıralama ve tanımsız davranış ... 10
3.10 Tip dönüşümleri: örtük, açık ve bitlerin kaderi ... 10
3.11 math.h, yuvarlama ve -lm ... 12
4. Çözümlü Örnekler: Örnek Merdiveni ... 13
5. Hata Avcısı ... 22
6. Mühendis Gözüyle ... 25
7. Mini-OBS Projesi: v0.4 ... 25
8. Alıştırmalar ve Atölye ... 26
8.1 Atölye etkinlikleri ... 26
8.2 Alıştırmalar ... 27
9. Kendini Değerlendirme ... 29
10. AP-2 Köprüsü ve Merak Kutusu ... 29
11. Özet, Kavram Haritası ve Terim Sözlüğü ... 29
11.1 Haftanın özeti ... 29
11.2 Büyük Sorunun yanıtı ... 30
11.3 Haftaya köprü: H5 Karar Yapıları ... 30
11.4 Terim sözlüğü (Bölüm 6.8 terim standardıyla uyumlu) ... 30
12. Kaynaklar ... 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Tek bir tip dönüşümü, yüz milyonlarca dolarlık bir roketi nasıl havada parçalayabilir?

H3'te veriyi **temsil etmeyi** öğrendik; bu hafta onu **işliyoruz**. Her C operatörü işlemcinin aritmetik–mantık birimine (ALU) giden bir komuta dönüşür: "ALU'nun dili". Doğru ifade yazmak için C'nin **gruplamasını** (öncelik), tipleri nasıl **dönüştürdüğünü** ve hiçbir şeyi garanti etmediği durumları (tanımsız davranış) bilmeliyiz; Büyük Soru'nun yanıtı buradadır (Bölüm 11.2).

> GERÇEK DÜNYADAN · **4 Haziran 1996, Kourou: Ariane 5'in ilk uçuşu**
>
> Ariane 5, 501 sayılı ilk uçuşunda Kourou'dan (Fransız Guyanası) kalktı; soruşturma kurulunun raporuna göre uçuş dizisi başladıktan yaklaşık 40 saniye sonra, yaklaşık 3700 m irtifada rotasından saptı, parçalandı ve patladı (Lions, 1996).
>
> Uçuş dizisinin başlangıcından (H0: ana motor ateşleme dizisinin başlangıcı) yaklaşık 36,7 saniye sonra, yani kalkıştan yaklaşık 30 saniye sonra, **eylemsizlik referans sistemleri** (SRI) yazılımında 64 bitlik kayan noktalı bir değer (yatay hızla ilişkili hizalama değeri; Horizontal Bias, **BH**) 16 bitlik **işaretli tamsayıya** dönüştürülürken sığmadı ve **işlenen hatası** (Operand Error) oluştu. Dönüşüm korumasızdı; Ariane 4'ten yeniden kullanılan yazılım, Ariane 5'in çok daha büyük yatay hızları için sınanmamıştı. İki SRI de durdu; zincirin geri kalanı Şekil 4.1'dedir (Lions, 1996). Yazılım Ada'daydı: aralık dışı dönüşüm işlenmeyen bir **istisna** üretti. Aynı dönüşüm C'de **tanımsız davranıştır** (Hata Avcısı 7).

Şekil 4.1
Ariane 5 (Uçuş 501) hata zinciri: tek bir korumasız tip dönüşümünden roketin kaybına (Lions, 1996)



SRI arızası: H0 + 36,7 s (kalkıştan ≈ 30 s sonra) · parçalanma: H0 + ≈ 40 s, ≈ 3700 m irtifada
H0: ana motor ateşleme dizisinin başlangıcı · Kaynak: Lions (1996), Inquiry Board raporu · Yazılım: Ada

1.1 Öğrenme çıktıları

ÖÇ 4.1 Aritmetik, atama, artırma/azaltma, ilişkisel ve mantıksal operatörleri doğru kullanır. *(Bölüm 3.2–3.6)*

ÖÇ 4.2 Operatör önceliği ve birleşme yönünü uygulayarak ifadelerin değerini elle hesaplar; ifade ağacı çizer. *(Bölüm 3.8–3.9)*

ÖÇ 4.3 Örtük (implicit) ve açık (explicit, cast) tip dönüşümlerinin sonuçlarını öngörür. *(Bölüm 3.10)*

ÖÇ 4.4 Bit düzeyi operatörlerle maskeleme, kaydırma ve bayrak işlemleri yapar. *(Bölüm 3.7; Örnek 9, 10, 12)*

ÖÇ 4.5 math.h kütüphane fonksiyonlarını kullanarak matematiksel formülleri koda aktarır. *(Bölüm 3.11; Örnek 7, 11)*

2. Geçen Haftadan: Değişkenlerden İfadelere

H3'te değişkenin adı, tipi, değeri ve adresi olduğunu; tamsayıların ikiye tümleyenle, kesirli sayıların IEEE 754 ile saklandığını gördük. Bu hafta değişkenleri **ifadelerde** birleştireceğiz. Önce üç soruyu yanıtlayın; yanıtları `H04_K12_geri_cagirma.c` programıyla doğrulayabilirsiniz.

> **TAHMİN ET · Geri çağırma: üç soru**
>
> - **S1.** `UINT_MAX`'a 1 eklenince ne olur; tanımlı mı? `INT_MAX + 1` neden farklıdır?
> - **S2.** `0.1 + 0.2 == 0.3` neden 0 verir? Kesirli sayılar nasıl karşılaştırılmalı?
> - **S3.** `%6.2f` ile 3.14159 nasıl yazılır?
>
> **Yanıtlar** (gerçek çıktı: `0 | 0 | [ 3.14]`). S1: işaretsiz aritmetik 2^{32} modunda tanımlıdır, sonuç 0; işaretli taşma ise tanımsızdır. S2: üç sayı da ikili tabanda tam temsil edilemez; farkın küçük bir eşikten küçük olup olmadığına bakılır. S3: alan genişliği 6, iki ondalık: `[ 3.14]`. İlk ikisinin "neden"ine bu hafta dönüşüm kurallarıyla döneceğiz.

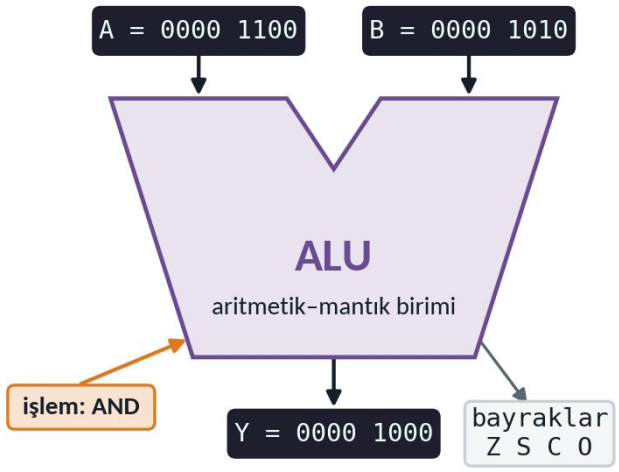
3. Kavramsal Anlatım

3.1 İfade, işlenen ve operatör: ALU'nun dili

Bir **ifade** (expression), bir değer hesaplayan **işlenenler** (operand) ve **operatörler** (operator) dizisidir: `a + b * 2`'de `a`, `b`, `2` işlenen; `+`, `*` operatördür. Operatörler **tekli** (unary: `-x`, `!x`, `~x`, `x++`), **ikili** (binary: `a + b`) ve **üçlü** (`k ? a : b`, H5) olabilir. Her ifadenin bir **tipi** ve **değeri** vardır (`7 / 2`: `int`, 3); `x = 5` ve `i++` gibi bazılarının ayrıca **yan etkisi** (side effect) vardır.

İfadeler işlemcinin **ALU**'suna (arithmetic logic unit) gider: iki ikili sayıyı alan, işlemi bir seçim sinyaliyle öğrenen ve sonucun yanında sıfır/işaret/elde/taşma **bayrakları** üreten devre (Patterson & Hennessy, 2017). `a & b` bir `and`, `a + b` bir `add` komutuna dönüşür; karşılaştırmalar `cmp` ile bayrakları ayarlar (Şekil 4.2). Çarpma ve bölme ayrı, daha yavaş devrelerdedir (Bölüm 3.7.3).

Şekil 4.2
ALU ve C operatörlerinin x86-64 makine komutlarındaki karşılıkları



C operatörü	x86-64 komutu	birim
`+ -`	`add sub`	ALU
`& | ^`	`and or xor`	ALU
`~`	`not`	ALU
`<< >>`	`shl shr/sar`	ALU (kaydırıcı)
`< == >=`	`cmp + bayrak`	ALU
`*`	`imul`	çarpma birimi
`/ %`	`idiv / div`	bölme birimi

Derleyici her operatörü bir ya da birkaç komuta çevirir.

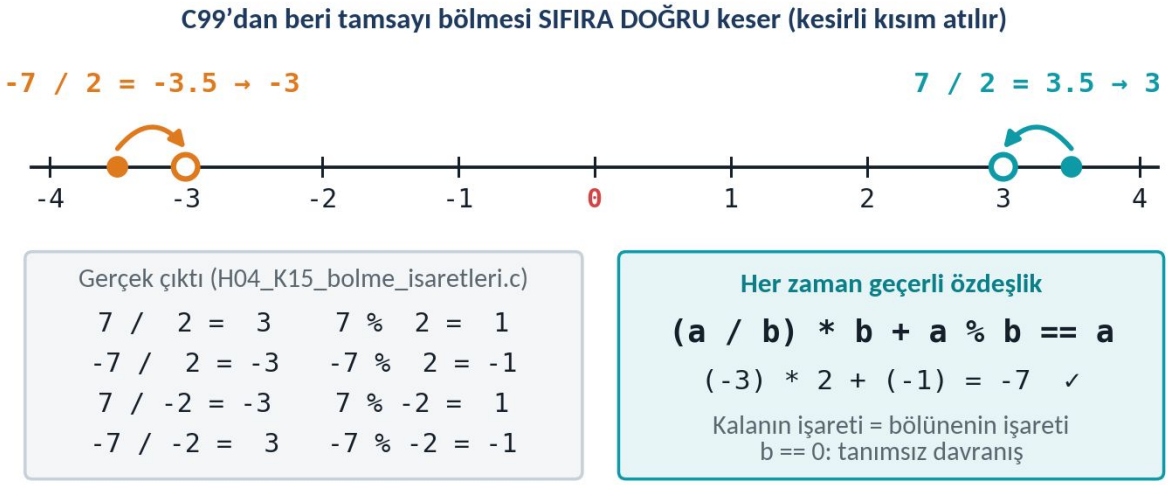
`12 & 10 = 8 → tek makine komutu: and`

Z: sonuç sıfır · S: işaret · C: elde · O: taşma

3.2 Aritmetik operatörler: tamsayı bölmesi ve kalan

Aritmetik operatörler `+ - * / %`'dir. **İki işlenen de tamsayıysa bölme tamsayı bölmesidir**, kesir atılır: `7 / 2` → 3; biri kayan noktalıysa `7.0 / 2` → 3.5 (Bölüm 3.10). `%` (kalan) yalnızca tamsayılarla çalışır (kesirli için `fmod`). **C99'dan beri tamsayı bölmesi sıfıra doğru keser**: −3,5 → −3. Kalan ise `(a / b) * b + a % b == a` özdeşliğini sağlayacak biçimde tanımlanır; bu yüzden **kalanın işareti bölünenin işaretidir**: `-7 % 2` → −1, `7 % -2` → 1 (Şekil 4.3; ISO/IEC, 2018). C'nin `%`'i matematikteki "mod" değil, bir kalan operatörüdür.

Şekil 4.3
Tamsayı bölmesi sıfıra doğru keser; kalan özdeşliği (sol alttaki değerler H04_K15_bolme_isaretleri.c programının gerçek çıktısıdır)



> **HATA AVCISI · Tamsayıda sıfıra bölme tanımsız davranıştır**
> `a / 0` için standart sonuç tanımlamaz. x86-64'te `idiv` sıfır bölenle donanım istisnası üretir, Linux programı **SIGFPE** ile sonlandırır: Örnek 1 b = 0 ile "Floating point exception" verip çöktü (çıkış kodu 136 = 128 + 8). Kayan noktalı bölmede IEEE 754 `inf`/`nan` üretir (H3). Önce denetim: kısa devreyle (3.6), H5'te `if` ile.

3.3 Atama ve bileşik atama

C'de `=` bir **operatördür**: sağdakini hesaplayıp soldaki değişkene yazar ve ifadenin değeri de budur. Atama **sağdan sola** birleşir: `x = y = z = 7;` önce `z`'ye yazar. Solda bir değişken (sol değer, lvalue) olmalıdır; `5 = x;` derlenmez. **Bileşik atama** (compound assignment) `x op= e`, `x = x op (e)` demektir: `+=`, `-=`, `*=`, `/=`, `%=` ve bit operatörleri için `&=`, `|=`, `^=`, `<<=`, `>>=`. Parantez önemlidir: `x *= 2 + 1` → `x = x * 3`.

`H04_K11_bilesik_atama.c` programında x = 20 ile başlayıp sırasıyla `x += 5`, `x -= 3`, `x *= 2`, `x /= 5`, `x %= 3`, `x *= 2 + 1` uygulandığında gerçek çıktı x'in 25, 22, 44, 8, 2 ve 6 değerlerinden geçtiğini gösterir (`x /= 5` tamsayı bölmesidir: 44 / 5 = 8).

3.4 Artırma ve azaltma: önek ve sonek

`++`/`--` bir değişkeni 1 artırır/azaltır; fark ifadenin **değerindedir**: **sonek** `i++`'nın değeri **eski** değerdir (artırma yan etki olarak olur), **önek** `++i` önce artırır, değeri **yeni** değerdir. `int a = i++;` (i = 5) → a = 5, i = 6; `int b = ++i;` → b = 7. Tek başına deyim olarak (`i++;`) fark yoktur (Örnek 5; aynı değişkeni bir ifadede iki kez değiştirmek tanımsızdır: 3.9).

3.5 İlişkisel operatörler ve C'de doğruluk değeri

`<`, `<=`, `>`, `>=`, `==` (eşit mi?) ve `!=` karşılaştırmanın sonucunu `int` tipinde **0 ya da 1** olarak üretir. Koşul olarak kullanılan değerlerde **0 yanlış, sıfırdan farklı her değer doğrudur**. `!` değeri 0/1'e çevirir: `!0` → 1, `!7` → 0; `!!x` "x sıfırdan farklı mı?" sorusunun 0/1 yanıtıdır. Sonuçlar sayı olduğu için aritmetikte kullanılabilir: `(x > 5) + (x > 6) + (x > 7)` x'in kaç eşikten büyük olduğunu sayar; Şubat'ın gün sayısı `28 + artik` olur (Örnek 8).

> **HATA AVCISI · Üç klasik tuzak**
>
> - **Atama mı, eşitlik mi?** `=` ile `==`: `(puan = 100)` bir atamadır; `puan`'ı değiştirir, değeri 100'dür (HA2; H5'te `if` ile).
> - **Zincirleme karşılaştırma:** `0 <= puan <= 100`, `(0 <= puan) <= 100` diye gruplanır ve **her zaman 1** olur; doğrusu `0 <= puan && puan <= 100` (Alıştırma 4.6).
> - **Kayan noktalı eşitlik:** `0.1 + 0.2 == 0.3` → 0 (H3); farkı bir eşikle karşılaştırın.

3.6 Mantıksal operatörler ve kısa devre değerlendirme

Koşullar `&&` (VE), `||` (VEYA) ve `!` (DEĞİL) ile birleştirilir; sonuç 0/1'dir (`H04_K02_iliskisel_mantiksal.c` doğruluk tablolarını yazdırır). `&&`, `||`'den **sıkı bağlar**: `a || b && c` = `a || (b && c)`; karışık kullanımda parantez yazın. İki güvence vardır: işlenenler **soldan sağa** değerlendirilir ve sonuç sol işlenenden belliyse **sağ işlenen hiç değerlendirilmez**: `A && B`'de A yanlışsa sonuç 0, `A || B`'de A doğruysa sonuç 1'dir. Buna **kısa devre değerlendirme** (short-circuit evaluation) denir. Görmek için sağ tarafa ekrana yazan bir `printf` koyalım:

H04_K03_kisa_devre.c · satır 13–25

```
13      int s1 = (x != 0) && printf("   sağ taraf çalıştı\n");
14      printf("   sonuç = %d\n", s1);
15
16      printf("2) (x == 0) && printf(...)\n");
17      int s2 = (x == 0) && printf("   sağ taraf çalıştı\n");
18      printf("   sonuç = %d\n", s2);
19
20      printf("3) (x == 0) || printf(...)\n");
21      int s3 = (x == 0) || printf("   sağ taraf çalıştı\n");
22      printf("   sonuç = %d\n", s3);
23
24      int b = 0;
25      int guvenli = (b != 0) && (10 / b > 1);   /* 10 / b hiç hesaplanmaz */
```

ÇIKTI

```
$ ./kisa_devre
1) (x != 0) && printf(...)
   sonuç = 0
2) (x == 0) && printf(...)
   sağ taraf çalıştı
   sonuç = 1
3) (x == 0) || printf(...)
   sonuç = 1
4) (b != 0) && (10 / b > 1) = 0
```

"sağ taraf çalıştı" yalnızca 2. durumda göründü: 1. durumda `x != 0` yanlış olduğundan `&&`, 3. durumda `x == 0` doğru olduğundan `||` sağ tarafı atladı. 2. durumda `printf` yazdığı bayt sayısını (sıfırdan farklı) döndürdüğü için sonuç 1'dir. 4. durum en yaygın kullanımdır: `(b != 0) && (10 / b > 1)`'de bölme yalnızca b ≠ 0 ise yapılır; **sıfıra bölme hiç olmaz**. Bit operatörleri `&` ve `|` kısa devre **yapmaz** (HA3).

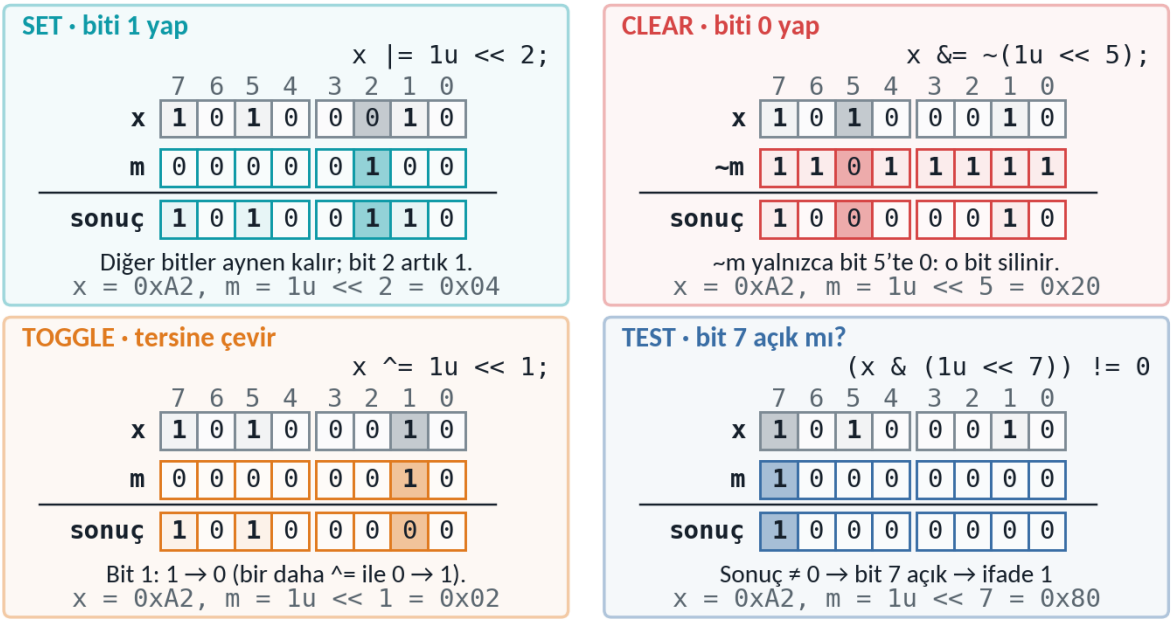
3.7 Bit düzeyi operatörler

Bit düzeyi operatörler bir tamsayının bitlerine **tek tek** işlem uygular: `&` (VE), `|` (VEYA), `^` (ÖZEL VEYA, XOR), `~` (bitleri tersine çevirir), `<<` ve `>>` (kaydırma). `&&` iki doğruluk değerinden 0/1 üretir; `&` ise 32 bitin her birini komşusundan bağımsız işler: `0xCA & 0x5C` = `0x48`, `0xCA | 0x5C` = `0xDE`, `0xCA ^ 0x5C` = `0x96` (`H04_K04_bitsel.c`'nin gerçek çıktısı). `unsigned int` 32 bit olduğu için `~0xCA` = `0xFFFFFF35`'tir; yalnızca alt 8 biti görmek için `& 0xFF` ile **maskeleriz**. Bit örneklerinde her zaman **işaretsiz** tipler kullanırız.

3.7.1 Maskeler ve bayraklar: set, clear, toggle, test

Bir tamsayının bitleri bağımsız **bayraklar** (flag) olabilir: 32 bitlik bir değişken 32 evet/hayır bilgisi taşır. Bitleri seçen değere **maske** (mask) denir; k'inci biti seçen maske `m = 1u << k`'dır. Dört temel işlem (Şekil 4.4): **set** `x |= m;` (`b | 1 = 1`, `b | 0 = b`), **clear** `x &= ~m;` (`~m` yalnızca o bitte 0'dır), **toggle** `x ^= m;` (`b ^ 1` tersidir, `b ^ 0 = b`) ve **test** `(x & m) != 0` (yalnızca o bit 1 ise sonuç sıfırdan farklıdır).

Şekil 4.4
Set, clear, toggle ve test: x = 0xA2 üzerinde bit şeritleri



Test ifadesinde parantez zorunludur: `!=`, `&`'ten sıkı bağlar; `x & m != 0` ifadesi `x & (m != 0)` olur (Alıştırma 4.5). Örnek 12 aynı işlemleri Linux dosya izinlerinde uygular.

3.7.2 Kaydırma: 2'nin kuvvetleriyle çarpma ve bölme

`x << k` bitleri sola kaydırıp sağdan 0 doldurur; `x >> k` sağa kaydırır. İşaretsiz sayılarda sola kaydırma **2^k ile çarpma**, sağa kaydırma **2^k'ye tamsayı bölmesi**dir: `5u << 3` → 40, `40u >> 3` → 5 (Örnek 10). Standardın üç kuralı (ISO/IEC, 2018): kaydırma miktarı negatifse ya da tipin bit genişliğine **eşit veya büyükse** davranış **tanımsızdır** (32 bitlik tipte `x << 32`); **negatif** işaretli değeri **sola** kaydırmak **tanımsızdır**; negatif işaretli değeri **sağa** kaydırmanın sonucu **uygulama tanımlıdır** (implementation-defined): GCC x86-64'te soldan işaret biti dolduran **aritmetik kaydırma** yapar.

ÇIKTI

```
$ ./kaydir_isaret      # H04_K05_isaretli_kaydirma.c, n = -17
n / 4    = -4   (bölme sıfıra doğru keser)
n >> 2   = -5   (gcc: aritmetik kaydırma)
-16 >> 2 = -4
n >> 2   = 0xFFFFFFFB  (soldan işaret biti 1 girer)
u >> 2   = 0x3FFFFFFB  (işaretsiz: soldan 0 girer)
```

`-17 / 4` → −4 (sıfıra doğru), ama `-17 >> 2` → −5 (aşağıya): işaretli sayılarda `>>` bölmenin yerine **geçemez**.

3.7.3 Kaputun altında: derleyici çarpmayı nasıl yapıyor?

`H04_K10_kaydirma_asm.c` dosyasındaki dört küçük fonksiyonu (`carp8`: `return x * 8;`, `bol8`: işaretsiz `x / 8`, `bol8_isaretli`: aynı bölme `int` ile, `tek_mi`: `x & 1`; fonksiyonlar H8'in konusu, burada yalnızca kalıp olarak) `gcc -O2 -S` ile derleyip çevirici (assembly) çıktısını okuduk:

GCC -O2 -S ÇIKTISI (KESİT)

```
$ gcc -std=c17 -O2 -S H04_K10_kaydirma_asm.c -o -      # kesit: .cfi/.size
yönergeleri çıkarıldı
carp8:
    endbr64
    leal 0(,%rdi,8), %eax
    ret
bol8:
    endbr64
    movl %edi, %eax
    shrl $3, %eax
    ret
bol8_isaretli:
    endbr64
    testl %edi, %edi
    leal 7(%rdi), %eax
    cmovns %edi, %eax
    sarl $3, %eax
    ret
```

> KAPUTUN ALTINDA • **Dört gözlem**
>
> - `x * 8` için çarpma komutu yok: `leal 0(,%rdi,8)` adres hesaplama devresiyle x × 8'i tek komutta bulur (`lea`); başka sabitlerde derleyici `sal`/`shl` da seçebilir. İşaretsiz `x / 8` → `shrl $3`.
> - İşaretli `x / 8` tek kaydırma değildir: negatif sayıya önce 7 eklenir (`leal 7(%rdi)`, `cmovns`), sonra `sarl $3`. Çünkü `>>` aşağıya yuvarlar, `/` sıfıra doğru keser; derleyici C'nin kuralını korur.
> - `tek_mi` için üretilen kod `andl $1` komutudur: en düşük anlamlı bit. (`endbr64` hesaplamayla ilgisiz bir güvenlik işaretidir.) Sonuç: `x * 8` yerine `x << 3` yazmak hızlandırmaz, yalnızca okunabilirliği azaltır; niyetinizi açık yazın.

3.8 Öncelik, birleşme yönü ve ifade ağaçları

`2 + 3 * 4` 14'tür, 20 değil. C, işlenenlerin nasıl **gruplanacağını** iki kuralla belirler: **öncelik** (operator precedence: `* / %` → `+ -` → kaydırma → ilişkisel → eşitlik → `&` → `^` → `|` → `&&` → `||` → atama) ve aynı

öncelikte **birleşme yönü** (associativity; Kernighan & Ritchie, 1988): aritmetik **soldan sağa** (`10 - 4 - 3` = 3; `100 / 10 / 5` = 2), tekli operatörler ve atama **sağdan sola** (Şekil 4.5).

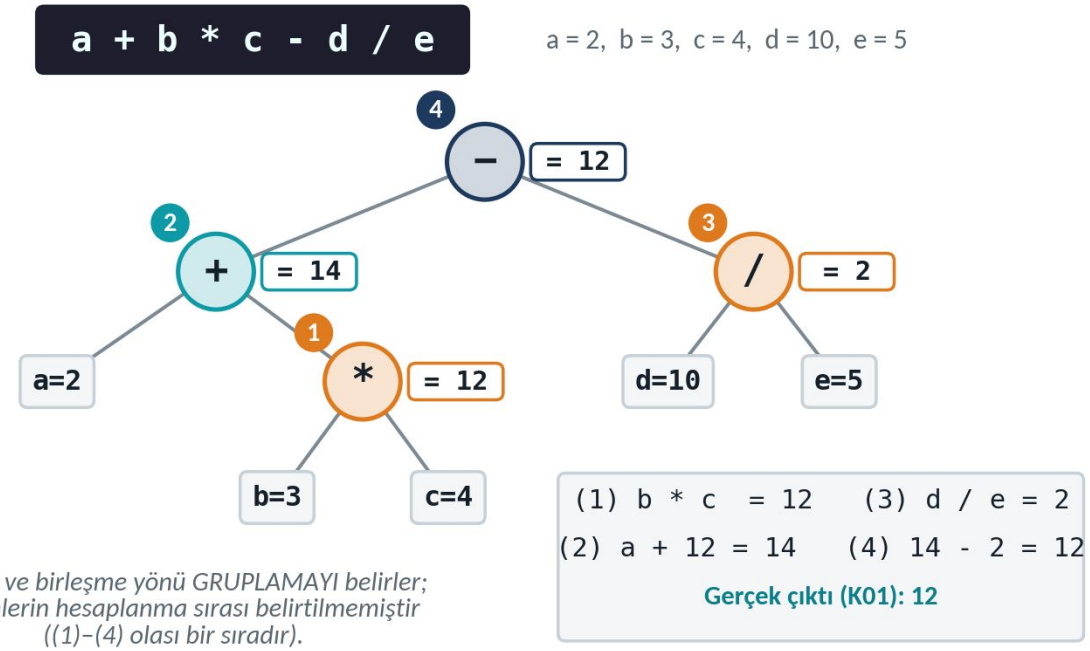
Şekil 4.5
C operatörlerinin öncelik tablosu (yukarıdan aşağıya azalan öncelik; renkler operatör gruplarını gösterir)

Düz.	Operatörler	Anlamı · yön
1	`() [] -> . x++ x--`	sonek, çağrı, dizi, üye
2	`++x --x +x -x ! ~ (tip) sizeof *p &x`	tekli (unary) **sağdan sola**
3	`* / %`	çarpma, bölme, kalan
4	`+ -`	toplama, çıkarma
5	`<< >>`	kaydırma
6	`< <= > >=`	ilişkisel
7	`== !=`	eşitlik
8	`&`	bit düzeyi VE
9	`^`	bit düzeyi ÖZEL VEYA
10	`|`	bit düzeyi VEYA
11	`&&`	mantıksal VE
12	`||`	mantıksal VEYA
13	`?:`	koşul (H5) **sağdan sola**
14	`= += -= *= /= %= &= |= ^= <<= >>=`	atama **sağdan sola**
15	`,`	virgül

Yön belirtilmeyen düzeyler soldan sağa. Üstteki satır daha sıkı bağlar; emin değilseniz parantez kullanın.

Gruplamayı görmenin yolu **ifade ağacı**dır (expression tree): yapraklar işlenen, iç düğümler operatör; en son uygulanan operatör köktedir. `a + b * c - d / e` → `(a + (b * c)) - (d / e)`; a = 2, b = 3, c = 4, d = 10, e = 5 için 12 (Şekil 4.6; değerler `H04_K01_oncelik.c`'nin gerçek çıktısıdır).

Şekil 4.6
*a + b * c - d / e ifadesinin ağacı ve düğüm değerleri*



Öncelik ve birleşme yönü GRUPLAMAYI belirler; işlenenlerin hesaplanma sırası belirtilmemiştir ((1)–(4) olası bir sıradır).

> NOT · **Gruplama ≠ değerlendirme sırası**
>
> Öncelik yalnızca hangi işlenenin hangi operatöre ait olduğunu belirler. `b * c` mi `d / e` mi önce hesaplanır? C bunu **belirtmez** (unspecified). Sıra güvencesi veren operatörler `&&`, `||`, virgül ve `?:`'dir. Pratik kural: aritmetikte matematik alışkanlığınız doğrudur; bit, kaydırma, karşılaştırma ve mantıksal operatörler karıştığında **parantez yazın**. `x & MASK == 0`, `1 << n + 1` (= `1 << (n + 1)`) ve `a + b / 2` (HA1) en sık öncelik hatalarıdır.

3.9 Yan etkiler, sıralama ve tanımsız davranış

Atama, `++`, `--` ve `printf` **yan etkilidir**. Standart, yan etkilerin sırasını yalnızca belirli noktalarda güvenceye alır: deyim sonu, `&&`, `||`, virgül ve `?:`'nin sol işleneni, fonksiyon çağrısı öncesi (C11: "önce sıralanır", sequenced before). **Aynı nesne sıralanmamış iki yan etkiyle değiştirilirse ya da biri değiştirirken diğeri okursa davranış tanımsızdır** (ISO/IEC, 2018). `i = i++ + ++i;` için GCC `-Wall` ile `operation on 'i' may be undefined [-Wsequence-point]`, Clang `multiple unsequenced modifications to 'i' [-Wunsequenced]` uyarır; `++n + ++n` bu makinede GCC ile **14**, Clang ile **13** verdi (HA5). İkisi de standarda uygundur: doğru soru "neden yazılmamalı?"dır.

Daha hafif bir tuzak **belirtilmemiş sıradır**: fonksiyon argümanlarının hesaplanma sırası belirtilmemiştir. `H04_K09_rand_zar.c` aynı tohumla (`srand(42)`) üç zarı önce üç ayrı deyimle (`int zar1 = rand() % 6 + 1;` …), sonra tek bir `printf`'te atıyor:

H04_K09_rand_zar.c · satır 18–20

```
18      srand(42);                      /* aynı tohum: aynı üç sayı yeniden */
19      printf("Tek printf: %d %d %d  (argüman sırası belirtilmemiştir)\n",
20             rand() % 6 + 1, rand() % 6 + 1, rand() % 6 + 1);
```

ÇIKTI

```
$ ./zar
Zarlar    : 1 1 6
RAND_MAX  : 2147483647
Tek printf: 6 1 1  (argüman sırası belirtilmemiştir)
```

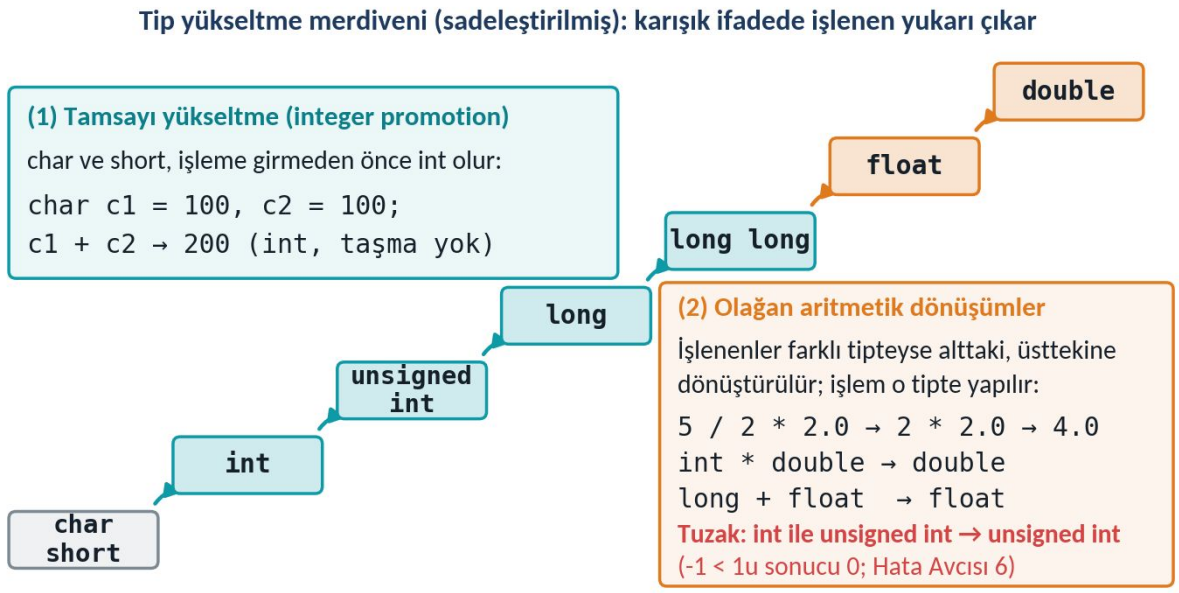
Aynı sayılar tek `printf`'te **ters sırada** geldi (GCC argümanları sağdan sola hesapladı). Kural: **bir değişkeni bir ifadede en çok bir kez değiştirin ve orada başka yerde okumayın; sıra önemliyse ayrı deyimler yazın.**

3.10 Tip dönüşümleri: örtük, açık ve bitlerin kaderi

Farklı tipler bir ifadede karşılaşınca C onları ortak tipe **dönüştürür**: programcı yazmadan **örtük** (implicit), `(tip)` yazarak **açık** (explicit, cast).

- **Tamsayı yükseltme** (integer promotion): `char` ve `short` işleme girmeden önce `int` olur; iki `char`'ın toplamı `int` toplamıdır (100 + 100 = 200, taşma yok).
- **Olağan aritmetik dönüşümler** (usual arithmetic conversions): iki işlenen farklı tipteyse alttaki üsttekine dönüşür (Şekil 4.7); kayan noktalı tipler tamsayılardan üstündür: `int * double` → `double`.
- **Atamada dönüşüm:** değer soldaki tipe dönüşür; `double` → `int` kesiri atar, `int` → `unsigned char` alt 8 biti bırakır.

Şekil 4.7
Tip yükseltme merdiveni (sadeleştirilmiş)



`H04_K06_yukseltme.c`'nin gerçek çıktısına göre `sizeof(c1)` 1, `sizeof(c1 + c2)` 4'tür (toplam `int`'tir); `sizeof('A')` da 4'tür, çünkü C'de karakter sabiti `int`'tir (C++'ta `char`). `u + 1` `int` olarak 256'dır, 1 baytlık `u`'ya yazılınca 0 kalır. Programın devamında `long + float` → `float`, `float * double` → `double` görülür.

> **HATA AVCISI · İşaretli ile işaretsiz karışınca: -1 < 1u yanlıştır**
>
> `int` ile `unsigned int` karşılaşınca işaretli değer **işaretsize** dönüşür: −1 (`0xFFFFFFFF`) 4 294 967 295 olur ve `-1 < 1u` **0** verir (HA6). GCC bunu yalnızca `-Wextra` ile uyarır: `comparison of integer expressions of different signedness [-Wsign-compare]`; yalnızca `-Wall` ile sessiz kaldı.

3.10.1 Açık dönüşüm (cast)

`(tip) ifade` bir **tekli** operatördür ve `*`, `/`'den sıkı bağlar; yeri sonucu değiştirir (Örnek 6'nın ilk satırları):

H04_06_bolme_donusum.c · satır 10–14

```
10      printf("7 / 2           = %d\n", 7 / 2);
11      printf("7.0 / 2         = %f\n", 7.0 / 2);
12      printf("7 / 2.0         = %f\n", 7 / 2.0);
13      printf("(double)7 / 2   = %f\n", (double)7 / 2);
14      printf("(double)(7 / 2) = %f\n", (double)(7 / 2));
```

ÇIKTI (İLK BEŞ SATIR)

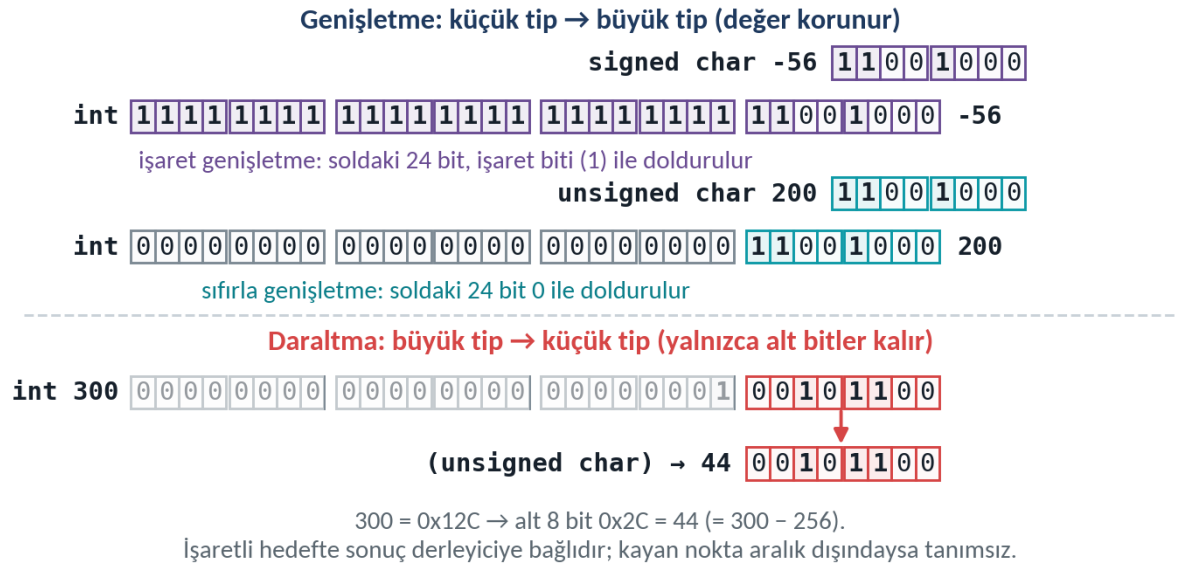
```
$ ./bolme
7 / 2           = 3
7.0 / 2         = 3.500000
7 / 2.0         = 3.500000
(double)7 / 2   = 3.500000
(double)(7 / 2) = 3.000000
```

`(double)7 / 2` → 7 önce double olur, bölme kayan noktalıdır: 3.5. `(double)(7 / 2)` → parantez içi önce tamsayı bölmesiyle 3 olur: kesir çoktan kaybolmuştur. Ortalamada doğru kalıp `(double)toplam / adet`'tir (Örnek 7).

3.10.2 Kaputun altında: genişletme ve daraltma

- **İşaret genişletme** (sign extension; Bryant & O'Hallaron, 2016): işaretli değerin üst bitleri işaret bitinin kopyasıyla dolar: `signed char` −56 (`1100 1000`) → `0xFFFFFFC8` = −56. **Sıfırla genişletme:** işaretsizde 0 ile: `unsigned char` 200 (aynı bitler) → `0x000000C8` = 200.
- **Daraltma** (narrowing): alt bitler kalır. Hedef işaretsizse tanımlıdır (2^n modunda): `(unsigned char)300` = 44. İşaretli hedefe sığmıyorsa sonuç **uygulama tanımlıdır**; GCC alt bitleri alır: `(short)70000` → 4464.
- **Kayan noktadan tamsayıya:** kesir **sıfıra doğru** atılır (`(int)-3.99` → −3); tam kısım hedefe **sığmıyorsa tanımsız davranış:** Ariane 5'in C'deki karşılığı (HA7).

Şekil 4.8
İşaret genişletme, sıfırla genişletme ve daraltma (değerler H04_K07_genisletme_daraltma.c programının gerçek çıktısı)



3.11 math.h, yuvarlama ve -lm

Karekök, üs, mutlak değer ve yuvarlama `<math.h>`'teki fonksiyonlarla yapılır; hepsi `double` alır ve döndürür: `sqrt(2.0)` → 1.414214, `pow(2.0, 10)` → 1024.0, `fabs(-2.7)` → 2.7, `floor(-2.7)` → −3.0 (aşağı), `ceil(-2.7)` → −2.0 (yukarı), `round(2.5)` → 3.0 (en yakına, yarımda sıfırdan uzağa) (`H04_K08_yuvarlama_math.c`'nin gerçek çıktısı). Linux'ta (glibc) bu fonksiyonlar ayrı bir kitaplıktadır (`libm`); `-lm` unutulursa derleme geçer, **bağlama** başarısız olur:

BAĞLAYICI HATASI (-LM UNUTULDU, ÖRNEK 11)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H04_11_ikinci_derece.c -o kokler
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
H04_11_ikinci_derece.c:16:(.text+0x83): undefined reference to `pow'
/usr/bin/ld: H04_11_ikinci_derece.c:17:(.text+0xbe): undefined reference to `sqrt'
/usr/bin/ld: H04_11_ikinci_derece.c:18:(.text+0x10f): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status
```

Bildirim `<math.h>`'te vardı, gövdeyi bulan olmadı (H1'deki derleme hattı). K08 ise altı fonksiyon çağırdığı hâlde `-lm` olmadan yalnızca `round` için hata verdi: sabit argümanlı çağrıları (`sqrt(2.0)`) GCC derlemede hesaplar (**sabit katlama**, constant folding). Windows'ta MinGW ve macOS'ta genellikle `-lm` gerekmez; eklemek zararsızdır.

Yuvarlama. Bir değeri tamsayıya çevirmenin yolları farklı sonuç verir (Şekil 4.9): `(int)x` sıfıra doğru keser; `round` yarımda sıfırdan uzağa gider; `printf("%.0f")` bu makinede (glibc) tam yarımda **çift olana** yuvarlar (2.5 → "2"); `(int)(x + 0.5)` hilesi yalnızca **negatif olmayan** sayılarda doğrudur (−2.7 → −2). Notlar negatif olamayacağı için Mini-OBS'de bu kalıbı kullanabiliriz.

Şekil 4.9
Yuvarlama yollarının karşılaştırması (H04_K14_yuvarlama_tablosu.c programının gerçek çıktısı; kırmızı: beklenmedik sonuç)

x	(int)x	floor(x)	ceil(x)	round(x)	(int)(x+0.5)	%.0f
2.5	2	2	3	3	3	**2**
-2.5	-2	-3	-2	-3	-2	-2
2.7	2	2	3	3	3	3
-2.7	-2	-3	-2	-3	**-2**	-3

trunc: sıfıra doğru · floor: aşağı · ceil: yukarı · round: en yakına, yarımda sıfırdan uzağa

printf %.0f (glibc): yarımda ÇİFTE yuvarlar (2.5 → 2)
(int)(x + 0.5) negatiflerde yanlış (−2.7 → −2)

rand/srand'a ilk bakış. `rand()` (`<stdlib.h>`) 0…`RAND_MAX` arasında sözde rastgele sayı üretir; `srand(tohum)` başlangıcı belirler; aynı tohum aynı diziyi verir (K09: 1, 1, 6), farklı dizi için `srand((unsigned)time(NULL));`. `rand() % 6 + 1` zar üretir. `RAND_MAX` burada 2147483647, Windows'un C kitaplığında 32767'dir.

4. Çözümlü Örnekler: Örnek Merdiveni

Her basamak bir yeni öge ekler (Tablo 4.1). Önce çıktıyı **tahmin edin**, sonra çalıştırın (PRIMM; Sentance vd., 2019). Kod blokları dosyadaki satır numaralarını taşır; başlık yorumu ve okuma satırları yer kazanmak için gösterilmez (tam dosyalar `kodlar/`). Çıktılar gerçektir; kullanıcının yazdığı değer istemin sonundadır. **Amiral örnekler:** 4 ve 12.

Tablo 4.1
Haftanın örnek merdiveni (dosyalar: kodlar/H04_01_… – H04_13_…)

#	Örnek	Yeni öge / amaç	Düzey
1	Dört işlem ve kalan operatörü	Tamsayı bölmesi ve %	★
2	Saniyeyi saat:dakika:saniye biçimine çevirme	/ ve % birlikte kullanımı	★
3	Üç basamaklı sayının basamaklarını ayırma ve basamak toplamı	%10 ve /10 deseni	★★
4	Para üstü: en az sayıda banknot/madeni para	Ardışık bölme–kalan; açgözlü yaklaşım sezgisi	★★
5	i++ ve ++i çıktı tahmini	Önek/sonek; tahmin et–çalıştır (PRIMM)	★★
6	7/2, 7.0/2 ve (double)7/2 karşılaştırması	Örtük ve açık dönüşüm	★★
7	Notların ortalamasında cast ve yuvarlama	Uygulamada tip dönüşümü; round	★★

#	Örnek	Yeni öge / amaç	Düzey
8	Artık yıl koşulunun tek bir ifade olarak yazılması	Mantıksal operatörler; sonucun 0/1 olması	★★
9	Tek/çift: % 2 ve & 1	Aynı sonuca iki yol; bit düzeyi düşünme	★★
10	2'nin kuvvetleri ile sola/sağa kaydırma	<<, >>; çarpma/bölme karşılığı	★★
11	İkinci dereceden denklemin diskriminantı ve kökleri	sqrt, pow, -lm (Δ < 0 denetimi H5'e köprü)	★★
12	Dosya izin bitleri: set/clear/toggle/test	Maskeler ve bayraklar	★★★
13	XOR ile takas (merak kutusu)	^ operatörü; neden pratikte tercih edilmez?	★★★

4.1 Örnek 1 ★ Dört işlem ve kalan operatörü

H04_01_dort_islem.c · satır 15–21

```
15      printf("a + b = %d\n", a + b);
16      printf("a - b = %d\n", a - b);
17      printf("a * b = %d\n", a * b);
18      printf("a / b = %d   (tamsayı bölmesi: kesir atılır)\n", a / b);
19      printf("a %% b = %d   (kalan)\n", a % b);
20      printf("(a / b) * b + a %% b = %d   (a ile aynı olmalı)\n",
21             (a / b) * b + a % b);
```

ÇIKTI

```
$ ./dort_islem      # negatif bölünen
İki tamsayı girin (a b): -7 2
a + b = -5
a - b = -9
a * b = -14
a / b = -3   (tamsayı bölmesi: kesir atılır)
a % b = -1   (kalan)
(a / b) * b + a % b = -7   (a ile aynı olmalı)
```

Yeni ögeler: iki `int`'in bölümü tamsayıdır; `%` kalanı verir; `printf`'te `%` yazdırmak için `%%`. −7 / 2 = −3, −7 % 2 = −1 ve özdeşlik sağlanıyor (17 5 ve 7 −2 çalıştırmaları `kodlar/ciktilar/`). b = 0 girildiğinde program `a * b` satırından sonra `Floating point exception` ile çöktü (tanımsız davranış; bu makinedeki gözlem). **Değiştir:** Programı `double` ile yazın; `%` neden derlenmez?

4.2 Örnek 2 ★ Saniyeyi saat:dakika:saniye biçimine çevirme

H04_02_saniye_donustur.c · satır 16–19

```
16      int saat = toplam_saniye / 3600;
17      int kalan = toplam_saniye % 3600;
18      int dakika = kalan / 60;
19      int saniye = kalan % 60;
```

ÇIKTI

```
$ ./saniye
Süre (saniye): 3725
3725 saniye = 01:02:05
```

`/` "kaç tam birim?", `%` "geriye ne kaldı?" sorusunu yanıtlar: 3725 / 3600 = 1 saat, kalan 125 → 2 dakika 5 saniye; `%02d` iki basamaklı, başa 0 ekleyen alandır. **Değiştir:** Gün sayısını ekleyip 100 000 saniyeyi "1 gün 03:46:40" yazdırın.

4.3 Örnek 3 ★★ Üç basamaklı sayının basamakları

H04_03_basamaklar.c · satır 16–19

```
16      int birler = sayi % 10;
17      int onlar = (sayi / 10) % 10;
18      int yuzler = sayi / 100;
19      int toplam = yuzler + onlar + birler;
```

ÇIKTI

```
$ ./basamak      # ortada sıfır
Üç basamaklı bir sayı: 905
Yüzler: 9  Onlar: 0  Birler: 5
Basamak toplamı: 14
```

`x % 10` son basamağı verir, `x / 10` son basamağı atar. Bu desen H6'da döngüyle her uzunlukta sayıya, H13'te kendi `atoi`'mize genişleyecek. **Değiştir:** Sayıyı tersine çevirin (Alıştırma 4.7).

4.4 Örnek 4 ★★ Para üstü: en az sayıda parça (amiral örnek)

1. Problem. Bir kasa, para üstünü **en az sayıda** banknot/madeni parayla verecek: 200, 100, 50, 20, 10, 5 ve 1 TL. **2. Çözümleme.** Girdi TL cinsinden negatif olmayan bir tamsayı (kuruşlu tutarı `double` okuyup 100 ile çarpmak H3'teki temsil hatalarını getirir; kuruşları tamsayı tutmak güvenlidir). Sınır durumları: 0, yalnızca 1 TL'ler (4), büyük tutar (1234).

Tablo 4.2
Örnek 4'ün IPO çözümlemesi

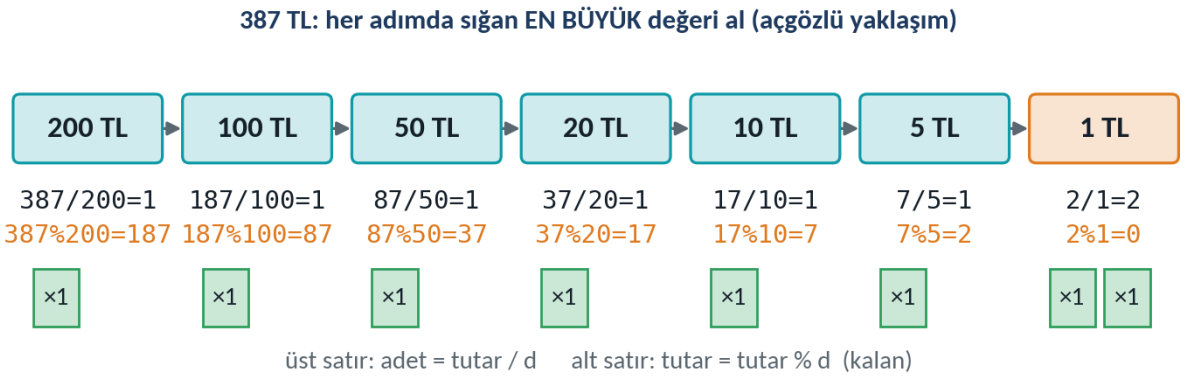
GİRDİ	İŞLEM	ÇIKTI
• `tutar` (TL, tamsayı ≥ 0)	• Her değer d için (büyükten küçüğe): • `adet = tutar / d` • `tutar %= d`	• Her değerden adet • Toplam parça

3. Algoritma (açgözlü yaklaşım). Her adımda **sığan en büyük değeri** mümkün olduğunca çok kullan, kalanla devam et (Şekil 4.10). Döngü henüz yok; her değer ayrı satırda (H6'da tek döngü olacak). **4. C kodu:**

H04_04_para_ustu.c · satır 16–20

```
16      int adet200 = tutar / 200;   /* önce en büyük değer */
17      tutar %= 200;                /* tutar = tutar % 200 */
18      int adet100 = tutar / 100;
19      tutar %= 100;
20      int adet50 = tutar / 50;
```

Şekil 4.10
387 TL için açgözlü para üstü: her adımda / ile adet, % ile kalan



5. Açıklama. Okuma satırları (10–14) ve yazdırma satırları (30–38) gösterilmedi; tutar, değiştirilmeden önce yazdırılır. 16. satırdaki tamsayı bölmesinin kesiri atması tam istediğimizdir (387 / 200 = 1). 17. satır bileşik atamadır: `tutar = tutar % 200` → 187; sonraki satırlar kalanla çalışır. 5'e bölümden kalan 0–4'tür ve 1 TL'lerle karşılanır (28. satır; gösterilmeyen satırlar aynı deseni 20, 10 ve 5 için sürdürür). **6. İzleme** (girdi 387): tutar 387 → 187 → 87 → 37 → 17 → 7 → 2; her adımda adet 1, son adımda `adet1` = 2. **7. Çalıştırma:**

```
ÇIKTI
$ ./para_ustu
Para üstü (TL, tamsayı): 387
387 TL için en az sayıda banknot/madeni para:
200 TL x 1
100 TL x 1
 50 TL x 1
 20 TL x 1
 10 TL x 1
  5 TL x 1
  1 TL x 2
Toplam parça: 8
```

8. Test. Gerçek çalıştırmalarda toplam parça sayıları: 387 → 8, 1234 → 12, 4 → 4 parça; beklenenlerle (200+100+50+20+10+5+1+1; 6×200+20+10+4×1; 4×1) aynı.

Açgözlü yöntem her zaman en iyisi mi? TL gibi kanonik sistemlerde evet; her sistemde değil (1–3–4 karşı örneği: AP-2 Köprüsü). **9. Değiştir – genişlet:** (a) 500 TL ekleyin. (b) Kuruşları (50, 25, 10, 5, 1) ekleyin; tutarı kuruş cinsinden `int` okuyun. (c) 1–3–4 sisteminde 6'yı programla doğrulayın.

4.5 Örnek 5 ★★ i++ ve ++i: önce tahmin et, sonra çalıştır

H04_05_artirma_azaltma.c · satır 12–26

```
12      int a = i++;    /* sonek: önce değeri kullan (5), sonra artır */
13      printf("a = i++  ->  a = %d, i = %d\n", a, i);
14
15      int b = ++i;    /* önek: önce artır (7), sonra değeri kullan */
16      printf("b = ++i  ->  b = %d, i = %d\n", b, i);
17
18      int c = i--;
19      printf("c = i--  ->  c = %d, i = %d\n", c, i);
20
```

```
    int d = --i;
    printf("d = --i  ->  d = %d, i = %d\n", d, i);

    i++;            /* tek başına deyim: önek ya da sonek fark etmez */
    ++i;
    printf("son      ->  i = %d\n", i);
```

ÇIKTI

```
$ ./artirma
a = i++  ->  a = 5, i = 6
b = ++i  ->  b = 7, i = 7
c = i--  ->  c = 7, i = 6
d = --i  ->  d = 5, i = 5
son      ->  i = 7
```

İncele: en sık tahmin hatası `c = i--` satırında c'ye 6 yazmaktır; sonek biçimde değişken önce **kullanılır**, sonra azalır. Son iki satırda değer kullanılmadığı için önek/sonek farkı yoktur (5 → 7). **Değiştir:** `int e = i++ * 2;` ekleyin; e ve i ne olur?

4.6 Örnek 6 ★★ 7/2, 7.0/2 ve (double)7/2

H04_06_bolme_donusum.c · satır 15–25

```
    printf("5 / 2 * 2.0     = %f\n", 5 / 2 * 2.0);
    printf("5 * 2.0 / 2     = %f\n", 5 * 2.0 / 2);
    printf("(int)3.99       = %d\n", (int)3.99);
    printf("(int)-3.99      = %d\n", (int)-3.99);

    int toplam = 7, adet = 2;
    double yanlis = toplam / adet;          /* önce int bölmesi: 3 */
    double dogru = (double)toplam / adet;   /* önce dönüşüm: 3.5 */
    printf("yanlis = %.2f, dogru = %.2f\n", yanlis, dogru);
    return 0;
}
```

Gerçek çıktı: `5 / 2 * 2.0` → 4.000000, `5 * 2.0 / 2` → 5.000000, `(int)3.99` → 3, `(int)-3.99` → −3, `yanlis = 3.00, dogru = 3.50`. `5 / 2 * 2.0` soldan sağa: önce `5 / 2` **tamsayı** bölmesiyle 2, sonra 4.0; `5 * 2.0 / 2` ilk işlemde double'a geçer: 5.0. `double yanlis = toplam / adet;` en yaygın hatadır: bölme `int` olarak yapılmıştır. **Değiştir:** `yanlis`'ı üç yolla düzeltin; hangisi en okunaklı?

4.7 Örnek 7 ★★ Not ortalamasında cast ve yuvarlama

H04_07_not_ortalamasi.c · satır 18–25

```
    double ortalama = (double)toplam / 4;     /* cast: kesirli bölme */

    printf("Toplam              : %d\n", toplam);
    printf("toplam / 4          : %d\n", tamsayi_ort);
    printf("(double)toplam / 4  : %.2f\n", ortalama);
    printf("round(ortalama)     : %.0f\n", round(ortalama));
    printf("(int)(ortalama + 0.5): %d\n", (int)(ortalama + 0.5));
    printf("printf ile %%.0f     : %.0f\n", ortalama);
```

ÇIKTI

```
$ ./not_ort      # ortalama tam 70.5
Dört quiz notu: 70 71 70 71
Toplam               : 282
toplam / 4           : 70
(double)toplam / 4   : 70.50
round(ortalama)      : 71
(int)(ortalama + 0.5): 71
printf ile %.0f      : 70
```

Ortalama **tam 70.5** iken `round` ve `(int)(ortalama + 0.5)` 71 verdi, `printf("%.0f")` 70 yazdı (yarımda çifte). "70.5 → 71" isteniyorsa `printf`'e güvenmeyin, değeri açıkça yuvarlayın. `round` için `-lm`. Diğer çalıştırmalar (86.25 → 86, 59.75 → 60) `kodlar/ciktilar/`'da.

4.8 Örnek 8 ★★ Artık yıl koşulu tek bir ifade olarak

H04_08_artik_yil.c · satır 15–21

```
15      int artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
16
17      printf("%d %% 4   == 0 : %d\n", yil, yil % 4 == 0);
18      printf("%d %% 100 != 0 : %d\n", yil, yil % 100 != 0);
19      printf("%d %% 400 == 0 : %d\n", yil, yil % 400 == 0);
20      printf("Artık yıl mı? (1 evet, 0 hayır): %d\n", artik);
21      printf("Şubat bu yıl %d gün çeker.\n", 28 + artik);
```

ÇIKTI

```
$ ./artik_yil      # 100'e bölünür, 400'e bölünmez
Yıl: 1900
1900 % 4   == 0 : 1
1900 % 100 != 0 : 0
1900 % 400 == 0 : 0
Artık yıl mı? (1 evet, 0 hayır): 0
Şubat bu yıl 28 gün çeker.
```

Yıl 4'e bölünüyorsa artıktır; 100'e bölünenler ise 400'e de bölünmüyorsa değildir. `&&` zaten `||`'den sıkı bağlar; parantez okunabilirliği artırır ve `-Wparentheses` uyarısını önler. Kısa devre sayesinde yıl 4'e bölünmüyorsa ikinci test yapılmaz. 2024 ve 2000 için sonuç 1, 1900 ve 2026 için 0'dır. **H5 köprüsü:** "artık yıl" / "değil" yazmak bir **karar** gerektirir: `if (artik)`.

4.9 Örnek 9 ★★ Tek mi, çift mi? % 2 ve & 1

H04_09_tek_cift.c · satır 15–20

```
15      printf("n %% 2         = %d\n", n % 2);
16      printf("n & 1         = %d\n", n & 1);
17      printf("n %% 2 != 0    = %d   (tek mi?)\n", n % 2 != 0);
18      printf("(n & 1) == 1  = %d   (tek mi?)\n", (n & 1) == 1);
19      printf("n %% 2 == 1    = %d   (dikkat: negatif tek sayıda 0)\n",
20             n % 2 == 1);
```

ÇIKTI

```
$ ./tek_cift      # negatif tek sayı
Bir tamsayı: -7
n % 2         = -1
n & 1         = 1
n % 2 != 0    = 1   (tek mi?)
(n & 1) == 1  = 1   (tek mi?)
n % 2 == 1    = 0   (dikkat: negatif tek sayıda 0)
```

İkili ağırlıkların (1, 2, 4, 8, …) yalnızca $2^0 = 1$ tektir; tekliği **en düşük anlamlı bit** belirler ve `n & 1` onu okur. `-7 % 2` −1 olduğundan `n % 2 == 1` negatif tek sayıda **yanlış** (0) verir; taşınabilir test `n % 2 != 0`'dır. `n & 1` ikiye tümleyende (C23'te zorunlu) negatiflerde de doğrudur.

4.10 Örnek 10 ★★ 2'nin kuvvetleri: sola ve sağa kaydırma

H04_10_kaydirma.c · satır 14–19

```
14      printf("x << 3   = %3u  (0x%02X)  x * 8\n", x << 3, x << 3);
15      printf("x >> 1   = %3u  (0x%02X)  x / 2 (kesir atılır)\n", x >> 1,
16             x >> 1);
17      printf("40u >> 3 = %3u  (0x%02X)  40 / 8\n", 40u >> 3, 40u >> 3);
18
19      unsigned int bayrak = 1u << 4;        /* yalnızca 4. bit 1 */
```

ÇIKTI

```
$ ./kaydirma
x        =   5  (0x05)
x << 1   =  10  (0x0A)  x * 2
x << 3   =  40  (0x28)  x * 8
x >> 1   =   2  (0x02)  x / 2 (kesir atılır)
40u >> 3 =   5  (0x05)  40 / 8
1u << 4  =  16  (0x10)
1u << 10 = 1024 (2 üzeri 10)
1u << 31 = 2147483648
```

`x << k` = $x \cdot 2^k$, `x >> k` = $x / 2^k$ (işaretsiz); `1u << k` yalnızca k'inci biti 1 olan **maskedir**. `1u << 31` işaretsiz olduğu için tanımlıdır; işaretli `1 << 31` `int`'e sığmaz ve tanımsızdır. **Değiştir:** `1u << 32` yazıp derleyin; GCC ne diyor?

4.11 Örnek 11 ★★ İkinci dereceden denklemin kökleri

H04_11_ikinci_derece.c · satır 16–18

```
16      double delta = pow(b, 2) - 4 * a * c;          /* Δ = b² - 4ac */
17      double kok1 = (-b + sqrt(delta)) / (2 * a);
18      double kok2 = (-b - sqrt(delta)) / (2 * a);
```

ÇIKTI

```
$ ./kokler      # Δ < 0: H5'te if ile denetlenecek
a, b, c katsayıları: 1 2 5
Δ  = -16.00
x1 = -nan
x2 = -nan
```

1 −3 2 için Δ = 1, x_1 = 2, x_2 = 1 (tüm çalıştırmalar `kodlar/ciktilar/`). `pow` ve `sqrt -lm` ister (unutulunca oluşan hata Bölüm 3.11'de). `(2 * a)` parantezi zorunludur: `/ 2 * a` önce 2'ye böler, sonra a ile **çarpar**. **Δ < 0** iken `sqrt` NaN döndürür, glibc `-nan` yazar: program çökmez ama anlamsız sonuç üretir. Köklerin gerçel olmadığını bildirmek için Δ'nın işaretine göre **karar** gerekir: H5'te `if (delta < 0)`. (`b * b`, `pow(b, 2)`'den hızlı ve kesindir; burada `pow`'u tanıtıyoruz.)

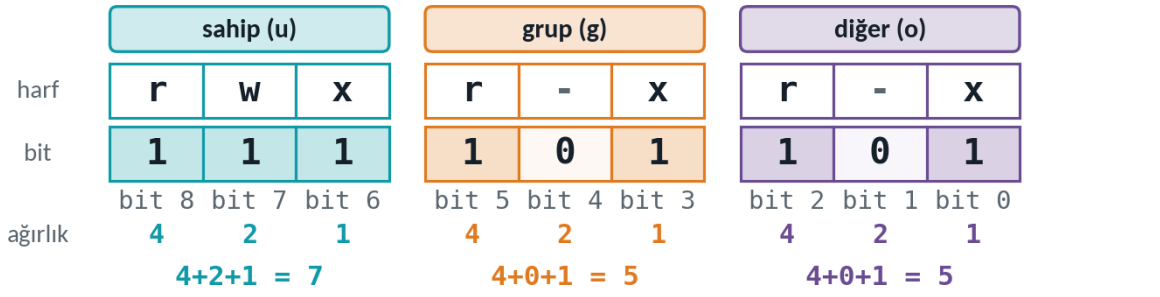
4.12 Örnek 12 ★★★ Dosya izin bitleri: set, clear, toggle, test (amiral örnek)

1. Problem. Linux'ta her dosyanın **sahip**, **grup** ve **diğerleri** için okuma (r), yazma (w), çalıştırma (x) izinleri vardır: 9 bit. `ls -l` bunları `rwxr-xr-x` gösterir, `chmod 755` aynı bitleri **sekizli** sayıyla ayarlar. İzinleri bir `unsigned int`'te bayrak olarak tutup set/clear/toggle/test yapalım. **2. Çözümleme.** Her sekizlik basamak 3 biti kodlar: r = 4, w = 2, x = 1; 7 = `rwx`, 5 = `r-x`, dolayısıyla 755 = `111 101 101` (Şekil 4.11). Bu ortamdaki gerçek komutlar:

UÇBİRİM (GERÇEK ÇIKTI; 644 VE 750 DENEMELERİ KODLAR/CİKTİLAR/'DA)

```
$ chmod 755 rapor.txt
$ stat -c '%A %a' rapor.txt
-rwxr-xr-x 755
```

Şekil 4.11
rwx izin bitleri: 111 101 101 = 7 5 5 = 0755



111 101 101 (ikili) = 7 5 5 (sekizli) = 0755 (C) = 493 (onlu)

> HATA AVCISI · **Baştaki 0: sekizlik taban tuzağı**
>
> C'de `0` ile başlayan tamsayı sabiti **sekizliktir**: `0755` = 7·64 + 5·8 + 5 = **493**; `755` ise yedi yüz elli beştir. Sayıları hizalamak için başa 0 koymak (`010` → 8) sinsi bir hatadır.

3. Algoritma. Maskeleri tanımla → izin = 0755 → grup yazmayı **test et** → grup yazmayı **aç** → diğerlerinin r ve x'ini **kapat** → sahibin x'ini iki kez **tersine çevir** → üç grubun basamaklarını kaydırma + maskeyle ayıkla. **4. C kodu:**

H04_12_dosya_izinleri.c · satır 11–29

```c
    const unsigned int SAHIP_R = 0400, SAHIP_W = 0200, SAHIP_X = 0100;
    const unsigned int GRUP_R = 0040, GRUP_W = 0020, GRUP_X = 0010;
    const unsigned int DIGER_R = 0004, DIGER_W = 0002, DIGER_X = 0001;

    unsigned int izin = 0755;            /* baştaki 0: sekizlik taban! */
    printf("Başlangıç            : %03o\n", izin);
    printf("Grup yazabilir mi?   : %d\n", (izin & GRUP_W) != 0);  /* test */

    izin |= GRUP_W;                      /* set: biti 1 yap */
    printf("izin |= GRUP_W       : %03o\n", izin);
    izin &= ~(DIGER_R | DIGER_X);        /* clear: bitleri 0 yap */
    printf("izin &= ~(R|X diğer) : %03o\n", izin);
    izin ^= SAHIP_X;                     /* toggle: tersine çevir */
    printf("izin ^= SAHIP_X      : %03o\n", izin);
    izin ^= SAHIP_X;                     /* bir kez daha: eski hâli */
    printf("izin ^= SAHIP_X      : %03o\n", izin);

    printf("Sahip/grup/diğer     : %o %o %o\n",
           (izin >> 6) & 7, (izin >> 3) & 7, izin & 7);
```

5. Açıklama. Her maske tek bir bitin sekizli yazımıdır (`0400` = bit 8, `0001` = bit 0). 17. satır: `0755 & 0020` = 0 → test 0. 19. satır: 0755 | 0020 = 0775. 21. satır: iki maske VEYA'lanır (`0005`), `~` yalnızca o bitleri 0 yapar, VE onları siler: 0770. Toggle kendi tersidir (0670 → 0770); `(izin >> 6) & 7` sahip basamağını okur.

Tablo 4.3
6. Örnek 12'nin izleme tablosu

Adım	İşlem	izin (ikili)	sekizli	Çıktı
1	`izin = 0755`	111 101 101	755	Başlangıç: 755
2	test `GRUP_W`	111 101 101	755	Grup yazabilir mi? 0
3	`izin |= GRUP_W`	**111 111 101**	775	775
4	`izin &= ~(DIGER_R|DIGER_X)`	**111 111 000**	770	770
5	`izin ^= SAHIP_X`	**110 111 000**	670	670
6	`izin ^= SAHIP_X`	**111 111 000**	770	770

7. ÇALIŞTIRMA (GERÇEK ÇIKTI)

```
$ ./izinler
Başlangıç            : 755
Grup yazabilir mi?   : 0
izin |= GRUP_W       : 775
izin &= ~(R|X diğer) : 770
izin ^= SAHIP_X      : 670
izin ^= SAHIP_X      : 770
Sahip/grup/diğer     : 7 7 0
Diğerleri okuyabilir mi? 0
rw-r--r-- = 644, rwxrwxrwx = 777
0755 onlu tabanda 493; 755 ise 755
```

8. Test. Programın `rw-r--r-- = 644` ve 0755/0644/0750 değerleri, yukarıdaki gerçek `chmod`/`stat` çıktılarıyla tutarlıdır; `0755` ve `755`'in onlu değerleri (493, 755) sekizlik tuzağını doğrular. **9. Değiştir:** (a) Herkes için salt okunur yapan tek satırı yazın. (b) İzni `rwxr-x---` biçiminde yazdırmak için ne gerekir? (Karar: H5.)

4.13 Örnek 13 ★★★ XOR ile takas (Merak Kutusu)

H04_13_xor_takas.c · satır 13–16

```
13      a ^= b;                         /* a = a ^ b */
14      b ^= a;                         /* b = b ^ (a ^ b) = eski a */
15      a ^= b;                         /* a = (a ^ b) ^ eski a = eski b */
16      printf("XOR takası : a = %d, b = %d\n", a, b);
```

Gerçek çıktı: `XOR takası : a = 25, b = 12` (başta a = 12, b = 25).

> **MERAK KUTUSU · x ^ x = 0 ve x ^ 0 = x: takas çalışır, ama neden tercih edilmez?**
>
> - **Aynı nesnede çalışmaz:** a ve b aynı nesneyse ilk adım onu sıfırlar: programın devamında `c ^= c;` üç kez uygulanınca 7 değil 0 kaldı. İki ad aynı nesneyi gösterebilir mi? İşaretçilerle evet: H12'de `takas(&x, &x)`.
> - **Hızlı değildir:** Modern derleyiciler geçici değişkenli takası zaten yazmaçlarda yapar; XOR'un üç adımı birbirini bekler. "XOR daha hızlıdır" inancının günümüz işlemcilerinde dayanağı yoktur.
> - **Okunabilirliği düşürür.** XOR'un gerçek kullanım alanları eşlik (parity) hesabı, basit şifreleme ve hata denetimidir.

5. Hata Avcısı

Hatalı programların hepsi bu ortamda derlenip çalıştırıldı; iletiler GCC 13 (ve karşılaştırma için Clang 18) çıktısıdır (`kodlar/hata_avcisi/`, hatalı ve düzeltilmiş sürümler). Yedi hatanın dördünde derleyici **hiç uyarmaz**.

Tablo 4.4
Hata–ileti–neden–çözüm tablosu

Hata	İleti / belirti	Neden	Çözüm
HA1: `a + b / 2`	(uyarı yok) 80 yerine 115	`/`, `+`'dan önce; tamsayı bölmesi	`(a + b) / 2.0`
HA2: `==` yerine `=`	(uyarı yok) puan = 100 olur	`=` atamadır, değeri 100'dür	`(puan == 100)`
HA3: `&&` yerine `&`	(uyarı yok) 2 & 1 = 0; `|` sağ tarafı çalıştırır	Bit operatörü 0/1 üretmez, kısa devre yok	`&&`, `||`
HA4: `pow` → `int`; `-lm`	`undefined reference to 'pow'`	libm bağlanmadı; double → int kesme	`-lm`; `double tutar`
HA5: `i = i++ + ++i`	`[-Wsequence-point]`	Sıralanmamış yan etkiler	Ayrı deyimler
HA6: `-1 < 1u`	`[-Wsign-compare]` (yalnızca `-Wextra`)	int, unsigned'a dönüşür	Aynı işaretli tip
HA7: `(short)40000.0`	(uyarı yok) derleyiciye göre farklı sayı	Aralık dışı float → int: tanımsız	Aralık denetimi / doyurma

5.1 HA1–HA3: operatörü doğru seçmek

hata_avcisi/H04_HA1_oncelik_ortalama.c · satır 11–11

```
11      double ortalama = vize + final / 2;     /* HATA: önce final / 2 */
```

HA1 · Öncelik. `vize + final / 2` = 70 + 45 = **115** (gerçek çıktı: `Ortalama: 115.00`). `/` önce bağlar ve üstelik tamsayı bölmesidir. Düzeltilmiş: `(vize + final) / 2.0` → 80.00.

hata_avcisi/H04_HA2_atama_esitlik.c · satır 11–11

```
11      int tam_puan = (puan = 100);    /* HATA: == yerine = (atama!) */
```

HA2 · = ile ==. Atama `puan`'ı bozar ve değeri 100'dür: `puan = 100, tam puan mı? 100`. GCC burada uyarmaz; aynı ifade `if` içinde olsaydı `-Wparentheses` uyarırdı (H5). Düzeltilmiş: `(puan == 100)` → 0.

hata_avcisi/H04_HA3_bitsel_mantiksal.c · satır 11–15

```
11      int ikisi_de = kitap & dergi;     /* HATA: & bit düzeyinde VE */
12      printf("İkisinden de var mı? %d\n", ikisi_de);
13
14      int x = 0;
15      int sonuc = (x == 0) | printf("sağ taraf da çalıştı\n"); /* HATA: | */
```

HA3 · & ile &&, | ile ||. `2 & 1` bitleri VE'ler: 0 ("ikisi de var mı?" sorusunun yanıtı 1 olmalıydı). `|` kısa devre yapmadığı için sağdaki `printf` çalıştı ve sonuç `1 | 26 = 27` oldu (26, yazılan bayt sayısı; Türkçe harfler UTF-8'de 2 bayttır). Gerçek çıktı: `İkisinden de var mı? 0 / sağ taraf da çalıştı / sonuc = 27`.

5.2 HA4 · pow sonucunun int'e atanması ve -lm

hata_avcisi/H04_HA4_pow_lm.c · satır 16–16

```
16      int tutar = anapara * pow(1 + faiz, yil);   /* HATA: kuruşlar kesilir */
```

-WCONVERSİON İLE (GERÇEK ÇIKTI)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -Wconversion H04_HA4_pow_lm.c -o ha4 -lm
H04_HA4_pow_lm.c:16:17: warning: conversion from ‘double’ to ‘int’ may change value
[-Wfloat-conversion]
```

İki hata var: `-lm` olmadan bağlama başarısız (`undefined reference to 'pow'`; aynı ileti Bölüm 3.11'de); `-lm` ile program çalışır ama 1157,625 TL `int`'e atanırken **1157**'ye kesilir (gerçek çıktı: `3 yıl sonra: 1157 TL`). `-Wall -Wextra` bunu görmez; `-Wconversion` uyarır. Ayrıca `pow` kayan noktalı sonuç döndürür: tam sonuç beklenen bir hesapta bile kitaplık 99,99999… döndürürse `int` 99 olur; tamsayı üslerde tamsayı çarpması ya da `round` kullanın. Düzeltilmiş sürüm `double tutar` ve `%.2f` ile 1157.63 yazar.

5.3 HA5 · Aynı ifadede bir değişkeni iki kez değiştirmek

hata_avcisi/H04_HA5_sirasiz_yan_etki.c · satır 10–14

```
10      i = i++ + ++i;          /* HATA: i aynı ifadede birden çok kez değişir */
11      printf("i = %d\n", i);
12
13      int n = 5;
14      int k = ++n + ++n;      /* HATA: n iki kez değişir; sıra belirsiz */
```

GCC 13 VE CLANG 18 (GERÇEK ÇIKTI)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H04_HA5_sirasiz_yan_etki.c -o ha5
H04_HA5_sirasiz_yan_etki.c:10:7: warning: operation on ‘i’ may be undefined [-
Wsequence-point]
H04_HA5_sirasiz_yan_etki.c:10:7: warning: operation on ‘i’ may be undefined [-
Wsequence-point]
H04_HA5_sirasiz_yan_etki.c:14:19: warning: operation on ‘n’ may be undefined [-
Wsequence-point]
$ ./ha5
i = 12
k = 14

$ clang -std=c17 -Wall -Wextra -pedantic H04_HA5_sirasiz_yan_etki.c -o ha5_x
H04_HA5_sirasiz_yan_etki.c:10:10: warning: multiple unsequenced modifications to 'i'
[-Wunsequenced]
H04_HA5_sirasiz_yan_etki.c:14:13: warning: multiple unsequenced modifications to 'n'
[-Wunsequenced]
2 warnings generated.
$ ./ha5_x
i = 12
k = 13
```

İki derleyici de uyardı; `++n + ++n` GCC ile 14, Clang ile 13 verdi (i'nin iki derleyicide de 12 çıkması rastlantıdır). Standart bu ifadelere anlam vermez; düzeltilmiş sürüm niyeti ayrı deyimlerle yazar.

5.4 HA6 · İşaretli–işaretsiz karşılaştırma

hata_avcisi/H04_HA6_isaret_karsilastirma.c · satır 12–14

```
12      int soguk_mu = sicaklik < esik;   /* HATA: -5 işaretsize dönüşür */
13      printf("-5 < 10 mu? %d\n", soguk_mu);
14      printf("-1 < 1u mu? %d\n", -1 < 1u);
```

Gerçek çıktı: `-5 < 10 mu? 0 / -1 < 1u mu? 0`. −5, `unsigned int`'e dönüşür (4 294 967 291) ve 10'dan büyük olur. `-Wextra` iki satır için de `comparison of integer expressions of different signedness: ‘int’ and ‘unsigned int’ [-Wsign-compare]` uyarısını verdi; yalnızca `-Wall` ile uyarı gelmedi. Düzeltme: iki işleneni aynı işaretli tipe getirmek (`int esik`).

5.5 HA7 · Aralık dışı kayan nokta → tamsayı (Ariane 5)

hata_avcisi/H04_HA7_ariane_donusum.c · satır 9–11

```
 9      double yatay_hiz = 40000.0;       /* varsayımsal değer: 16 bite sığmaz */
10
11      short bh = (short)yatay_hiz;      /* HATA: aralık dışı dönüşüm */
```

AYNI KOD DÖRT BİÇİMDE DERLENDİ (GERÇEK ÇIKTILAR; CLANG -O2 DEĞERİ ÇALIŞTIRMADAN ÇALIŞTIRMAYA DEĞİŞİR)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H04_HA7_ariane_donusum.c -o ha7 -lm
$ ./ha7
double 40000.0 -> short -25536

$ gcc -std=c17 -Wall -Wextra -pedantic -g -fsanitize=float-cast-overflow
H04_HA7_ariane_donusum.c -o ha7_x
```

```
$ ./ha7_x
H04_HA7_ariane_donusum.c:11:5: runtime error: 40000 is outside the range of
representable values of type 'short int'
double 40000.0 -> short -25536

$ gcc -std=c17 -O2 H04_HA7_ariane_donusum.c -o ha7_x
$ ./ha7_x
double 40000.0 -> short 32767

$ clang -std=c17 -O2 H04_HA7_ariane_donusum.c -o ha7_x
$ ./ha7_x
double 40000.0 -> short 312205000
```

40000, `short`'a (−32768…32767) sığmaz; C'de bu **tanımsız davranıştır**: `gcc -O0` −25536, `gcc -O2` 32767, `clang -O2` bir `short`'a sığamayacak bir çöp değer verdi (bu değer çalıştırmadan çalıştırmaya değişir); yalnızca çalışma zamanı denetleyicisi (`-fsanitize=float-cast-overflow`) hatayı raporladı. −25536'nın kaynağı: `-O0`'da değer önce 32 bitlik `int`'e çevrilir (`cvttsd2si`: 40000 = `0x00009C40`) ve alt 16 bit (`0x9C40`, işaret biti 1) saklanır (`movw`; `kodlar/hata_avcisi/ciktilar/H04_HA7_ariane_donusum_asm.txt`). Düzeltilmiş sürüm `yatay_hiz >= SHRT_MIN && yatay_hiz <= SHRT_MAX` ile sığıp sığmadığını 0/1 hesaplar ve `fmin`/`fmax` ile değeri aralığa **sıkıştırır** (doyurma, saturation): `short 32767`. H5'te bu denetimi `if` ile yapacağız.

6. Mühendis Gözüyle

Ariane 5 raporu hatanın tek bir satırdan çok bir **süreç** sorunu olduğunu vurgular (Lions, 1996): Ariane 4'te doğru çalışan kod yeni rokette sınanmamış **varsayımlarla** (yatay hızın 16 bite sığacağı) kullanılmış; bazı dönüşümler korunmuş, bu dönüşüm korumasız bırakılmış; hizalama işlevi kalkıştan sonra Ariane 5'te işe yaramadığı hâlde Ariane 4'ten kalan bir gereksinimle çalışmayı sürdürmüştü. C programcısı için dersler:

- **Daraltan her dönüşümü sorgula** (`double` → `int`, `int` → `short`): değer her zaman sığıyor mu, sığmazsa ne olmalı? SEI CERT C kuralları FLP34-C (kayan nokta dönüşümü hedef aralıkta olmalı), INT31-C (tamsayı dönüşümü veri kaybetmemeli), INT34-C (geçersiz kaydırma) ve EXP30-C (yan etkilerin sırasına güvenme) bu haftanın hatalarını doğrudan kapsar (Carnegie Mellon University Software Engineering Institute, 2016).
- **Uyarıları aç, cast ile susturma:** `-Wall -Wextra` HA6'yı, `-Wconversion` HA4'ü yakaladı; HA7'deki cast ise `-Wconversion`'ı bile susturdu. Testte `-fsanitize=undefined,float-cast-overflow` kullan.
- **Parantez ücretsizdir; bir ifadeye bir yan etki; uygulama tanımlı davranışa** (negatif `>>`, `RAND_MAX`, `-lm`) **güvenme; mikro-optimizasyon yapma:** derleyici `x * 8`'i zaten `lea`'ya çeviriyor.
- **Taşmayı öngör:** `(dusuk + yuksek) / 2` büyük değerlerde taşar; `dusuk + (yuksek - dusuk) / 2` güvenlidir (AP-2, ikili arama).

7. Mini-OBS Projesi: v0.4

v0.3 tek öğrencinin notlarını okuyup basit ortalamayı yazdırıyordu (`91_Proje_MiniOBS/v0.3/`). **v0.4'ün yenilikleri:**

- **Ağırlıklı ortalama** (vize %40, final %60) ve ölçütler `const` sabitlerle: `VIZE_AGIRLIK = 0.40`, `FINAL_AGIRLIK = 0.60` (`double`), `GECME_ORTALAMA = 60`, `FINAL_BARAJI = 50` (`int`).
- **Yuvarlama:** `int yuvarlanmis = (int)(ortalama + 0.5);` (notlar negatif olamayacağı için geçerli; `round` alternatifi Bölüm 3.11).
- **Geçme koşulu tek mantıksal ifade:** `int gecti = (yuvarlanmis >= GECME_ORTALAMA) && (final >= FINAL_BARAJI);` `if` ve `?:` henüz yok; rapor satırı: `Geçme durumu (1=geçti, 0=kaldı): 1`. Başlık ve çerçeve v0.3 ile aynı kalır: "MİNİ-OBS v0.4 · Öğrenci Not Sistemi".

Kabul ölçütleri: sıfır uyarı; dört `const` sabit; ortalama iki ondalıkla, yuvarlanmış değer tamsayı, geçme durumu 0/1 raporlanır; aşağıdaki testleri geçer.

Tablo 4.5
Mini-OBS v0.4 test senaryoları ve referans çözümün gerçek sonuçları

Test	Vize / final	Amaç	Ortalama → yuvarlanmış	Geçti
T1	70 / 85	normal: geçer	79.00 → 79	1
T2	100 / 45	ortalama yeterli, final barajı altı	67.00 → 67	0
T3	73 / 50.5	sınır: ortalama tam 59.5 → 60	59.50 → 60	1
T4	72.5 / 88	kesirli notlar	81.80 → 82	1
T5	57 / 61	sınır: 59.4 → 59	59.40 → 59	0
T6	95 / 50	sınır: final tam 50	68.00 → 68	1
T7	1 / 98.5	kayan nokta tuzağı: 59.4999… → 59	59.50 → 59	0

> MİNİ-OBS PROJESİ · **T7: iki haftanın dersi bir arada**
>
> Vize 1, final 98.5 için ortalama matematikte tam **59.5**'tir; referans çözüm `Ortalama : 59.50` yazdığı hâlde **59** ve **0** buldu: 0.40 ve 0.60 ikili tabanda tam temsil edilemez (H3), hesaplanan değer 59.499999999999993'tür (`H04_K16_miniobs_t7.c`, `%.17g`) ve cast 59'a keser. Sınırlamayı belgeliyoruz; önce iki ondalığa yuvarlamak (`round(ortalama * 100) / 100`) gibi çözümleri sonraki sürümlerde tartışacağız. **v0.5 (H5):** harf notu, geçti/kaldı metni, girdi doğrulama.

8. Alıştırmalar ve Atölye

8.1 Atölye etkinlikleri

1. **İfade değerlendirme yarışması (20 dk):** web sayfasındaki tip dönüşümü tahmin kartları ve ifade ağacı çözümleyicisiyle bireysel tahmin → eşli tartışma → çalıştırma (kart yanıtları `H04_K13_tahmin_kartlari.c` ile derleyiciye doğrulatıldı).
2. **Bit maskesi atölyesi (20 dk):** 8 LED'lik sanal panelde (web) ve C'de LED yakma/söndürme/tersleme; Alıştırma 4.14.
3. **Formülden koda (30 dk):** serbest düşme ve bileşik faiz (Alıştırma 4.9–4.10). `1 / 2 * g * t * t` neden hep 0'dır?

8.2 Alıştırmalar

Alıştırmalarda verilen kodlar `kodlar/alistirmalar/` klasöründedir (dosya adları alıştırmanın içinde). Çözümler öğrenci sürümünde yer almaz.

Alıştırma 4.1 ★☆☆ *Çıktı tahmini*

Program çalıştırılmadan her satırın ne yazdıracağını bulun.

alistirmalar/H04_A01_cikti_tahmini_1.c · satır 9–12

```
 9      printf("%d %d %d\n", 17 / 5, 17 % 5, -17 / 5);
10      printf("%d %d\n", -17 % 5, 17 % -5);
11      printf("%d\n", 2 + 3 * 4 % 5);
12      printf("%d\n", (2 + 3) * 4 % 5);
```

Alıştırma 4.2 ★★☆ *Çıktı tahmini*

Bir izleme tablosu çizerek x, y ve z'nin son değerlerini bulun.

alistirmalar/H04_A02_cikti_tahmini_2.c · satır 9–13

```
 9      int x = 10;
10      int y = x++ * 2;
11      int z = --x + y;
12      x *= 3 - 1;
13      printf("x = %d, y = %d, z = %d\n", x, y, z);
```

Alıştırma 4.3 ★★☆ *Çıktı tahmini*

Her satırın tipini ve değerini bulun; son satırda öncelik tablosuna bakın.

alistirmalar/H04_A03_donusum_tahmini.c · satır 9–14

```
 9      printf("%.2f\n", 9 / 4 * 1.0);
10      printf("%.2f\n", 9 * 1.0 / 4);
11      printf("%d\n", (int)9.99 + (int)-9.99);
12      printf("%d\n", 'a' - 'A');
13      printf("%d\n", 3 > 2 && 2 > 1);
14      printf("%u\n", 0x0Fu << 4 | 0x03u);
```

Alıştırma 4.4 ★☆☆ *Hata bul*

Program 212 °F için 100.0 yerine 0.0 yazıyor. Neden? Tek karakter ekleyerek düzeltin.

alistirmalar/H04_A04_fahrenheit_hata.c · satır 14–14

```
14      double c = 5 / 9 * (f - 32);
```

Alıştırma 4.5 ★★☆ *Hata bul*

0644 izninde sahip yazabilir, ama program 0 yazıyor. GCC'nin `-Wall` uyarısını okuyun ve düzeltin.

alistirmalar/H04_A05_izin_oncelik_hata.c · satır 9–11

```
 9      unsigned int izin = 0644;
10
11      int sahip_yazabilir = izin & 0200 == 0200;
```

Alıştırma 4.6 ★★☆ *Hata bul*

150 puan geçersiz olduğu hâlde program 1 yazıyor. Neden? GCC'nin iki uyarısını yorumlayın.

alistirmalar/H04_A06_aralik_zinciri_hata.c · satır 9–11

```
 9      int puan = 150;
10
11      int gecerli = 0 <= puan <= 100;
```

Alıştırma 4.7 ★★☆ *Parsons*

Satırları doğru sıraya dizerek üç basamaklı bir sayıyı tersine çeviren programın gövdesini elde edin (472 → 274). Hiçbir satır eklenmez, silinmez.

alistirmalar/H04_A07_parsons_ters_cevir.c · satır 9–16

```
 9      printf("Tersi: %d\n", ters);
10      int yuzler = sayi / 100;
11      scanf("%d", &sayi);
12      int ters = birler * 100 + onlar * 10 + yuzler;
13      int birler = sayi % 10;
14      int sayi = 0;
15      int onlar = (sayi / 10) % 10;
16      printf("Üç basamaklı bir sayı: ");
```

Alıştırma 4.8 ★★☆ *Kod tamamla*

Dört boşluğu (`___`) doğru bit operatörüyle doldurun. Program sonunda ne yazdırır?

alistirmalar/H04_A08_led_tamamla.c · satır 9–14

```
 9      unsigned int led = 0x00;                /* 8 LED, hepsi sönük */
10
11      led = led ___ (1u << 3);                /* (a) 3. LED'i yak */
12      led = led ___ (1u << 7);                /* (b) 7. LED'i yak */
13      led = led ___ (1u << 3);                /* (c) 3. LED'i tersine çevir */
14      int yanik7 = (led ___ (1u << 7)) != 0;  /* (d) 7. LED yanıyor mu? */
```

Tablo 4.6
Kod yazma alıştırmaları (verilen kod yok; çözümler öğretim üyesinde)

No	Düzey	Görev
4.9	★	**Serbest düşme:** süreyi (s) okuyup düşülen yolu (h = ½gt², g = 9.81) ve son hızı (v = gt; m/s ve km/sa) yazdırın. `1 / 2` yazmanın tuzağını açıklayın.
4.10	★★	**Bileşik faiz:** A = P(1 + r/n)^{nt} ve kazanç (`pow`, `-lm`); 10 000 TL, %45, aylık, 2 yıl?
4.11	★★	**Yılın gün sayısı:** yılın kaç gün, Şubat'ın kaç gün çektiğini `if` kullanmadan yazdırın (artık yıl ifadesi 0/1'dir).
4.12	★★	**Süre farkı:** iki saat (sa dk sn) arasındaki süre `sa:dk:sn` ve saniye olarak (09:45:30 → 11:20:05).
4.13	★★★	**RGB ayrıştırma:** `%x` ile okunan `0xRRGGBB`'yi kaydırma ve maskeyle R, G, B'ye ayırın; gri tonu `0xGGGGGG` olarak paketleyin (`FF8800`, `0E9AA7`).
4.14	★★★	**8 LED'lik panel:** `led = 0`; 0. ve 5. LED'i yakın, hepsini tersine çevirin, 7.'yi söndürün; sonucu 8 bit ve yanan LED sayısı olarak (döngüsüz) yazdırın.

9. Kendini Değerlendirme

- ☐ `-7 / 2`, `-7 % 2` ve `7 % -2` ifadelerinin değerlerini gerekçesiyle söyleyebiliyorum; tamsayıda sıfıra bölmenin tanımsız olduğunu biliyorum.
- ☐ Bir biti set/clear/toggle/test eden ifadeleri yazabiliyor, `rwxr-xr-x` ↔ `0755` dönüşümünü yapabiliyorum.
- ☐ `7 / 2`, `(double)7 / 2`, `(double)(7 / 2)`, `'A' + 1` ifadelerinin tipini ve değerini öngörebiliyorum; `-1 < 1u` tuzağını açıklayabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

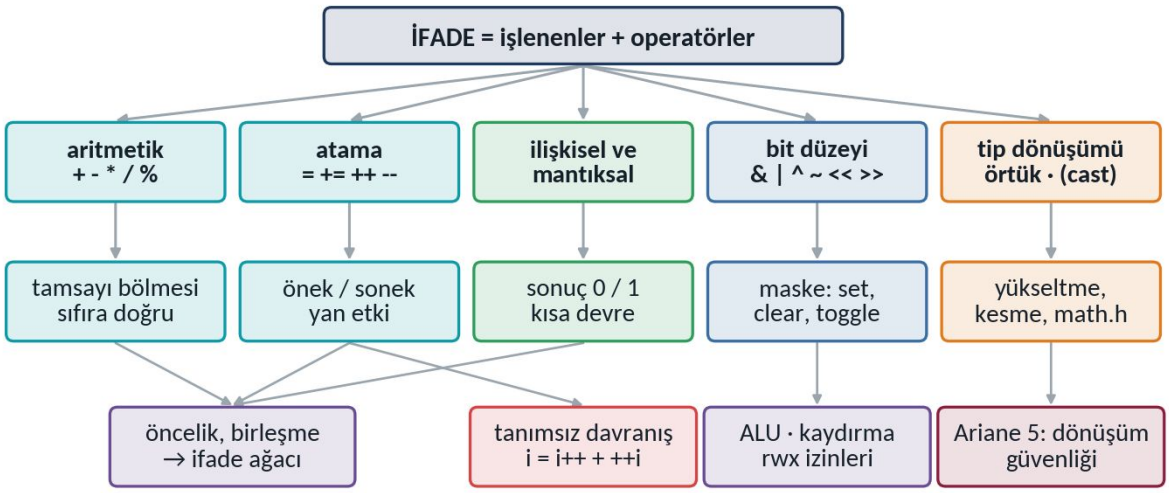
> **AP-2 KÖPRÜSÜ · Bu haftanın yapı taşları AP-2'de nerede?**
>
> - **Bit maskeleri:** n elemanlı kümenin 2^n alt kümesi 0…2^n − 1 sayılarının bitleriyle temsil edilir; `(maske >> k) & 1` "k'inci eleman var mı?" sorusudur. AP-2'deki alt küme üretimi buna dayanır.
> - **Açgözlü yaklaşım:** Para üstü (Örnek 4) açgözlü (greedy) algoritmaların ilk örneğidir; {1, 3, 4} ile 6'da başarısız olması (4 + 1 + 1 yerine 3 + 3) AP-2'de doğruluk kanıtı ihtiyacını doğurur.

11. Özet, Kavram Haritası ve Terim Sözlüğü

11.1 Haftanın özeti

- Tamsayı bölmesi sıfıra doğru keser, `%` bölünenin işaretini alır; ilişkisel/mantıksal sonuçlar 0/1'dir, `&&`/`||` kısa devre yapar; maskeler `|=`, `&= ~`, `^=`, `& … != 0`; öncelik gruplamayı belirler, sırayı değil.
- Yükseltme ve olağan aritmetik dönüşümler; cast'in yeri sonucu değiştirir; int ile unsigned karışınca −1 büyük olur; daraltma alt bitleri bırakır, aralık dışı float → int tanımsızdır (Ariane 5).

Şekil 4.12
Haftanın kavram haritası



Mini-OBS v0.4: ağırlıklı ortalama + (int)(x + 0.5) + (ort ≥ 60) && (final ≥ 50)

11.2 Büyük Sorunun yanıtı

Tek bir tip dönüşümü bir roketi nasıl parçalayabilir? Dönüşüm bir değerin temsilini değiştirir ve dar bir tipe her değer sığmaz. Ariane 5'te 64 bitlik bir değer 16 bitlik işaretli tamsayıya sığmadı; Ada'da istisna üretti, istisna işlenmediği için iki yönelim bilgisayarı da durdu ve roket kaybedildi (Lions, 1996). C'de aynı dönüşüm tanımsızdır (HA7: −25536, 32767 ya da çöp). Ders: **dönüşümden önce aralığı denetleyin**.

11.3 Haftaya köprü: H5 Karar Yapıları

Artık yıl mı (Örnek 8), Δ < 0 mı (Örnek 11), b sıfır mı (Örnek 1), öğrenci geçti mi (Mini-OBS) sorularını **0/1 ifadelerle** yanıtladık, ama program bu yanıtlara göre **farklı yol izleyemedi** (Δ < 0'da `-nan`, b = 0'da çökme). H5'te `if`, `else` ve `switch` bu ifadeleri yol ayrımlarına dönüştürecek.

11.4 Terim sözlüğü (Bölüm 6.8 terim standardıyla uyumlu)

Tablo 4.7
Haftanın terimleri

Türkçe	İngilizce
ifade; işlenen; operatör	expression; operand; operator
operatör önceliği; birleşme yönü	operator precedence; associativity
kısa devre değerlendirme	short-circuit evaluation
bit düzeyi operatör; maske; bayrak	bitwise operator; mask; flag
yan etki; sıralama noktası	side effect; sequence point
tanımsız / uygulama tanımlı / belirtilmemiş davranış	undefined / implementation-defined / unspecified behavior
tip dönüşümü (cast); örtük dönüşüm	type casting; implicit conversion
tamsayı yükseltme; olağan aritmetik dönüşümler	integer promotion; usual arithmetic conversions

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

Lions, J. L. (1996). *ARIANE 5 flight 501 failure: Report by the inquiry board*. European Space Agency.

Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design RISC-V edition: The hardware/software interface*. Morgan Kaufmann.

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education*, *29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781